QI Studio

AI Agentic Hackathon 2026

Problem Statements & Judging Rubric

Platform Engineering — Internal Intern Hackathon

Tagline: *Turn bold ideas into autonomous AI systems that solve real problems*
Theme: Build real-world multi-agent AI systems that can understand, coordinate, and act across complex systems.

Overview

This hackathon challenges interns to build real agentic AI systems — not demos, not toy prompts. Each team picks one of the five problem statements below and delivers a working proof-of-concept within the allotted time. Submissions are evaluated on technical depth, design quality, and the clarity of the live demonstration.

Duration	48 hours (single sprint)
Team size	2 – 3 interns per team
Deliverables	GitHub repo + Presentation and 10-minute live demo + a short README + Application URL
Languages	Any — Python and TypeScript/Node.js recommended
Infrastructure	Local or free-tier cloud (Azure, Vercel, Railway, etc.)

PROBLEM STATEMENT 1

Intent Router & Specialized Agent Dispatcher

Background

Modern AI assistants receive wildly varied requests: write code, fetch data, summarise a document, generate a report, answer a factual question. Routing every prompt through one generic LLM call is wasteful and imprecise. The better architecture is a thin routing layer that classifies the intent, selects the right specialised agent, and assembles a coherent response — all transparent to the end user.

Problem statement

Build an intent-routing system that accepts a raw natural-language prompt, classifies it, dispatches it to the appropriate specialised agent, and returns a unified response. The system must handle multi-domain prompts — where a single input requires more than one specialised agent — by decomposing the prompt and fanning out.

Required capabilities

- A classifier layer that identifies intent type, required tools, and ambiguity level from the raw prompt
- At least three distinct specialised agents (e.g. a data/SQL agent, a document/summarisation agent, a web-search/retrieval agent)
- Fan-out logic: a multi-domain prompt must be split, dispatched in parallel, and the partial responses merged into one coherent reply
- A confidence threshold: prompts below threshold must trigger a clarification request rather than a best-guess dispatch
- Instrumentation: the system must log which agent handled each sub-task and the routing decision rationale

Stretch goals

- Dynamic agent registration — add a new agent without touching the router code
- Streaming response — each sub-agent's partial output streams as it arrives
- A minimal UI showing the routing graph in real time

Constraints

- The router itself must not be a generic LLM call with a long system prompt — it must produce structured classification output (JSON)
- All external LLM calls must go through a single abstraction layer (no raw OpenAI SDK calls scattered across files)

Suggested stack

Orchestration	LangGraph, CrewAI, or custom state machine
Classifier	Any LLM
Agents	One per domain — tool-calling or ReAct pattern
API layer	FastAPI or Express
Observability	Console JSON logs minimum; LangSmith optional
Frontend	React

What a strong submission looks like

- The router correctly classifies 8 out of 10 test prompts provided on demo day
- A multi-domain prompt (e.g. "fetch the latest NVDA price and summarise today's earnings call") is visibly split and both agents run
- The README explains the classification schema and routing algorithm clearly
- The team can explain what happens when the classifier is wrong
- React based chat interface to verify output

PROBLEM STATEMENT 2

Multi-Agent Customer Support Resolution Engine

Background

Customer support teams handle thousands of requests every day across email, chat, voice transcripts, and ticketing systems. Not all issues are the same—some require product troubleshooting, some require billing investigation, while others need policy validation or escalation. Routing every customer issue through a single support workflow increases resolution times and reduces customer satisfaction.

Problem statement

Build a multi-agent customer support resolution engine that automatically analyzes incoming customer requests, identifies the issue type, coordinates specialized agents to investigate the problem, and generates a recommended resolution. The system must support complex tickets that require multiple departments or knowledge sources.

Required capabilities

- Intent classification of incoming support requests
- Ticket decomposition into sub-problems when multiple issues are present
- At least three specialized agents (e.g., Technical Support Agent, Billing Agent, Policy Agent)
- Multi-agent collaboration for complex customer cases
- Automatic severity and priority assessment
- Escalation handling for unresolved or high-risk cases
- Unified customer-facing response generation
- Audit trail showing investigation steps and agent contributions

Stretch goals

- Integration with either Linear, Zendesk, Freshdesk, or Jira Service Management
- Sentiment analysis and customer frustration detection
- Suggested knowledge-base article generation
- Real-time agent collaboration dashboard

Constraints

- Every resolution recommendation must include reasoning
- All agent interactions must be logged
- Support conversations should maintain context across multiple interactions

Suggested stack

Orchestration	LangGraph, CrewAI, AutoGen
Agents	Technical Support, Billing, Policy, Escalation
Backend	FastAPI or Express
Frontend	React
Observability	LangSmith, Open Telemetry, JSON Logs

What a strong submission looks like

- Correctly classifies support tickets across multiple categories
- Handles a ticket involving both technical and billing issues
- Displays agent collaboration flow
- Produces a customer-ready resolution summary

PROBLEM STATEMENT 3

Meeting Intelligence & Action Item Orchestrator

Background

Organizations spend significant time in meetings, but key decisions, action items, risks, and follow-ups are often lost. Team members manually create notes and track action items, resulting in missed commitments and poor accountability.

Problem statement

Build a meeting intelligence platform that transforms meeting transcripts into structured outputs including summaries, decisions, action items, risks, owners, deadlines, and follow-up recommendations. The system must use multiple specialized agents to analyze different aspects of the meeting.

Required capabilities

- Ingest meeting transcripts or recordings
- Automatically identify discussion topics
- Generate concise meeting summaries
- Extract action items with assigned owners
- Identify risks, blockers, and dependencies
- Detect decisions made during the meeting
- Create follow-up tasks and reminders
- Provide confidence scores for extracted information

Required agents

- Meeting Summarization Agent
- Action Item Extraction Agent
- Decision Detection Agent
- Risk & Dependency Analysis Agent

Stretch goals

- Live meeting analysis
- Integration with Jira, Asana, Linear, or ClickUp
- Calendar follow-up automation
- Speaker attribution and participation analytics

Constraints

- Must provide traceability from output back to transcript segments
- Must support meetings involving multiple topics
- All outputs should be structured JSON and human-readable summaries

Suggested stack

Orchestration	LangGraph
Speech-to-Text	Deepgram, Whisper
Backend	FastAPI or NodeJS server
Frontend	React

What a strong submission looks like

- Accurately extracts action items and owners
- Handles meetings involving product, engineering, and business discussions
- Shows which agent identified each output
- Creates actionable follow-up plans

PROBLEM STATEMENT 4

Contract Review & Risk Detection Agent

Background

Legal, procurement, and compliance teams spend significant effort reviewing contracts for risks, missing clauses, compliance violations, payment terms, renewal conditions, and liability exposure. Manual review is time-consuming and often inconsistent.

Problem statement

Build an AI-powered contract review system that analyzes contracts, identifies legal and business risks, highlights problematic clauses, and generates a risk assessment report. The platform should coordinate multiple specialized agents to review contracts from different perspectives.

Required capabilities

- Upload and process contracts in PDF or DOCX format
- Extract and classify contract clauses
- Identify risky language and unusual terms
- Detect missing standard clauses
- Assess financial, operational, and legal risks
- Generate executive-friendly risk summaries
- Provide clause-level explanations and recommendations

Required agents

- Clause Extraction Agent
- Legal Risk Agent
- Compliance Review Agent
- Financial Obligation Agent
- Executive Summary Agent

Fan-out logic

A contract must be analyzed simultaneously by multiple agents, with findings merged into a unified risk report.

Stretch goals

- Contract comparison against organization templates
- Regulatory compliance validation
- Contract redlining suggestions
- Risk scoring dashboard

Constraints

- Every identified risk must reference the source clause
- Risk recommendations must be explainable

- Review results must be auditable

Suggested stack

Orchestration	LangGraph, CrewAI
Document Processing	PyMuPDF, Unstructured
Backend	FastAPI
Frontend	React

What a strong submission looks like

- Correctly identifies high-risk clauses
- Explains risk rationale clearly
- Generates executive and legal review reports
- Displays agent-level findings and confidence scores

PROBLEM STATEMENT 5

AI Project Manager for Engineering Teams

Background

Engineering managers spend substantial time coordinating sprint planning, tracking dependencies, managing risks, monitoring team progress, preparing status reports, and identifying blockers. Much of this work involves collecting information from multiple tools and stakeholders.

Problem statement

Build an AI Project Manager that acts as an autonomous coordination layer for software engineering teams. The system should analyze project artifacts, identify risks, track delivery progress, generate status reports, and proactively recommend actions to improve project execution.

Required capabilities

- Ingest data from Jira, GitHub, Linear, Slack, and meeting notes
- Track sprint progress and delivery status
- Identify blockers and project risks
- Detect dependency conflicts
- Generate stakeholder status reports
- Recommend corrective actions
- Forecast sprint completion likelihood
- Monitor engineering productivity trends

Required agents

- Sprint Analysis Agent
- Risk Detection Agent
- Dependency Tracking Agent
- Stakeholder Reporting Agent
- Delivery Forecasting Agent

Fan-out logic

A project update request must trigger parallel analysis across multiple project dimensions, with outputs merged into a unified project health report.

Stretch goals

- Autonomous sprint planning recommendations
- Resource allocation optimization
- Capacity planning
- Natural language project queries

Constraints

- Recommendations must be supported by project data
- Historical trends must be considered during forecasting
- Every risk must include evidence and impact assessment

Suggested stack

Orchestration	LangGraph, CrewAI
Integrations	GitHub, Jira, Slack, Linear
Backend	FastAPI
Frontend	React
Observability	LangSmith

What a strong submission looks like

- Produces an accurate project health score
- Identifies hidden project risks and blockers
- Generates executive-ready status reports
- Demonstrates multi-agent collaboration across engineering data sources
- Explains why a project is on track or at risk with supporting evidence

Judging Rubric

Each team is scored out of 30 points (10 criteria × 3 points max). Scores are summed across all judges and averaged. In case of a tie, the team with the higher score on the Technical Depth criterion wins.

Scoring scale

0 — Insufficient	Not attempted, broken, or does not meet the minimum requirement
1 — Developing	Partially implemented; works in some cases but has significant gaps
2 — Competent	Fully implemented; works reliably for the core use case
3 — Excellent	Fully implemented with evidence of deeper thinking, edge case handling, or extensibility

Problem rubric — applies to all five statements

Criterion	Weight	0 – Insufficient	1 – Developing	2 – Competent	3 – Excellent
Intent / request classification	15%	Misclassifies most inputs; no routing logic	Correct on simple inputs; fails on ambiguous or mixed ones	Classifies the majority of inputs correctly and routes accordingly	High accuracy with confidence scoring and a clear fallback path
Multi-agent decomposition & fan-out	15%	Single monolithic flow; no decomposition	Decomposes but runs sequentially or leaves sub-tasks unmerged	Splits work across agents in parallel and merges the results	Merged output is demonstrably better than any single agent's result
Agent specialisation	10%	All agents are identical generic LLM calls	Agents differ but share the same tools and prompts	Each agent has distinct tools, prompts, and responsibilities	Agents are swappable; adding one requires no changes to existing code
Output quality & accuracy	10%	Output is incoherent or unrelated to the input	Output is partially correct but misses key items	Output is accurate and complete for the core use case	Output is accurate, structured, and includes confidence or evidence
Observability & traceability	10%	No logging or traceability	Logs exist but are unstructured	Structured logs show which agent handled each sub-task	Full trace or dashboard linking every output back to its source
Technical depth & code quality	10%	Monolithic; no separation of concerns	Some structure; key logic is still entangled	Clear layer separation; readable without explanation	Clean abstractions; a new agent or capability can be added in < 30 lines
Demo clarity	10%	Team cannot explain what the system does	Demo works but team struggles to explain choices	Demo is clear; team answers questions confidently	Demo shows a compelling real-world use case end-to-end
README	5%	Missing or placeholder	Covers setup only	Covers setup, architecture, and design decisions	Includes a decision log and known limitations
Edge case handling	10%	Crashes on unexpected input	Handles common errors; crashes on edge cases	Handles malformed input, API failures, and empty results	Graceful degradation with user-visible error context
Stretch goal bonus	5%	None attempted	Partially attempted	One stretch goal working	Two or more stretch goals working

General notes for judges

- Score what you see working during the demo, not what the team says will work later
- A smaller system that is fully working scores higher than an ambitious system that is mostly broken
- Ask each team member at least one technical question; individual understanding is part of the assessment
- Bonus points are capped — a team cannot exceed 30 total points

Submission checklist

- GitHub repo link shared before demo slot
- README includes: setup steps, architecture diagram or description, known limitations
- Demo environment is ready before the slot — no live debugging during demo time
- Each team member can explain their individual contribution